

Unidad Didáctica 3

Git y GitHub

(parte I)

Ciclo de Desarrollo de Aplicaciones Web

Módulo: Entornos de Desarrollo

Fray Paulino s/n 33600 MIERES – ASTURIAS Tfno. 985464982 Fax 985452969 / www.ibq.es ibq@ibq.es
C.I.F.: Q-3368149E

Contenido

1- Introducción.....3
2- Instalación y Configuración de Git y GitHub.....5
2.1. En cuanto a Git.....5
Configuración inicial de Git.....6
2.2. En cuanto a GitHub.....8
ANEXO - Autenticación de dos factores (2FA).....9
Configuración inicial de nuestro perfil en GitHub.....9
3- Creación de repositorios en GitHub.....12

1- Introducción

➢ ¿Qué es Git?

Git es una herramienta de control de versiones.

Permite controlar el proceso de creación de software llevando un **historial de los cambios que decidimos registrar mediante commits**. Con Git podemos:

- crear versiones del proyecto
- ramificar el desarrollo en diferentes flujos de trabajo (ramas o branches)
- volver atrás si algo sale mal (revertir cambios o commits).

Ejemplo: Imagina que estás desarrollando una web con varios compañeros. Uno de ellos cambia el archivo *index.html* y rompe el menú. Gracias a Git puedes volver exactamente al punto anterior en el que todo funcionaba.

Git es un programa **libre y gratuito** que se distribuye mediante la licencia GNU (GPL 2.0)

NOTA: *En realidad se puede extender el uso de Git a proyectos no solo de software sino también de documentos (hay muchos libros escritos con Git), presentaciones y, en general, a casi cualquier tipo de contenido*.

➢ ¿Qué es GitHub?

GitHub es una plataforma web que permite alojar proyectos controlados mediante Git.

Podemos pensar en GitHub como el "espejo" en la nube (en la red) en el que duplicar los proyectos que tenemos en nuestro propio ordenador (en local). Además, es una **red social para programadores**, donde los usuarios pueden:

- ver y descargar proyectos de otros
- dar "estrellas" ⭐ a los repositorios que les gustan
- seguir a otros desarrolladores
- colaborar en proyectos comunes mediante *pull requests* e *issues*.

Ejemplo: Un alumno sube su proyecto de clase a GitHub. Otro compañero puede hacerle una sugerencia o corregirle un error a través de una *pull request*. Esto simula perfectamente cómo se trabaja hoy en día en las empresas tecnológicas.

Hoy en día GitHub es tan influyente que muchas empresas revisan el perfil de GitHub de los candidatos durante los procesos de selección. Por eso se le suele llamar *"el escaparate del programador"*.

Su uso es gratuito para los **proyectos públicos**, sin límite en cuanto a la cantidad de repositorios.

➢ ¿Por qué utilizar Git junto con GitHub?

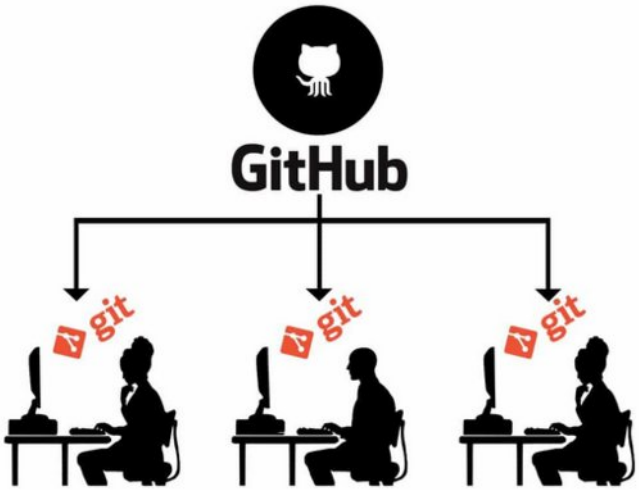
Usar Git por sí solo ya nos permite llevar un control de versiones muy preciso en nuestro ordenador.

Sin embargo, al combinarlo con GitHub, ganamos algo fundamental: **colaboración y trabajo en equipo**.

Ventajas principales

- **Copia de seguridad en la nube:** Todo el código queda almacenado online, de forma que si se rompe o se pierde el ordenador, el proyecto sigue a salvo en GitHub.
- **Trabajo en equipo sin caos:** Varias personas pueden modificar el mismo proyecto sin pisarse el trabajo gracias a las ramas (*branches*) y a los *merge* controlados.

  Ejemplo: mientras uno programa el sistema de *login*, otro puede trabajar en el diseño sin interferir.



- **Historial completo de cambios:** Git mantiene un registro de quién cambió qué y cuándo (mediante *commits*), lo que facilita detectar errores o retroceder a versiones anteriores.
- **Revisiones y sugerencias** (*pull requests*): GitHub permite que otros revisen tu código antes de incorporarlo al proyecto principal. Una Pull request es una propuesta para incorporar cambios a un proyecto.

  Ejemplo: igual que un profesor revisa un trabajo antes de aprobarlo, los compañeros revisan el código antes de integrarlo.
- **Visibilidad y reputación profesional:** GitHub funciona también como una red social. Tener un perfil activo con proyectos propios es una carta de presentación para empresas o equipos de desarrollo.

En resumen:

- ➢ **Git** = herramienta para **controlar versiones** de un proyecto.
- ➢ **GitHub** = **plataforma web** donde alojar repositorios Git para compartir proyectos y colaborar con otros desarrolladores.

2- Instalación y Configuración de Git y GitHub

2.1. En cuanto a Git

Para instalar Git en sistema Linux (por ejemplo, Ubuntu), basta con teclear la siguiente línea en una ventana de terminal:

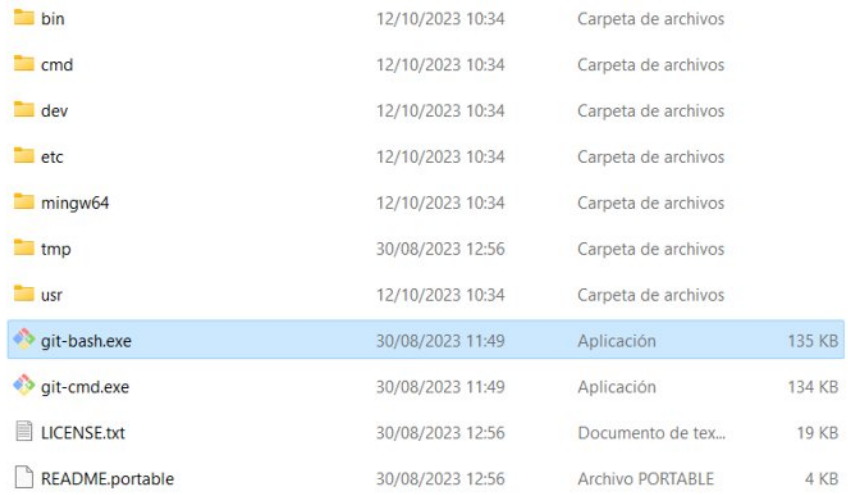
```
sudo apt install git
```

Para instalar Git en Windows, puedes consultar los binarios disponibles en https://git-scm.com/install/

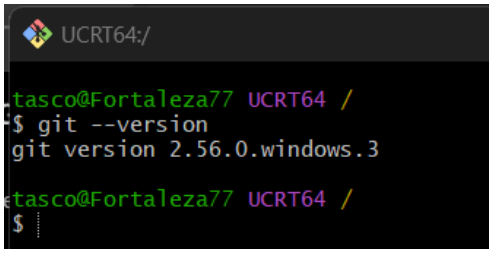
- En este curso, utilizaremos la versión portable para Windows que puedes encontrar en el curso del Aula Virtual). No requiere instalación: basta con descomprimir la carpeta y ejecutar **`git-bash.exe`** (para trabajar con una terminal Bash, similar a las utilizadas habitualmente en sistemas Linux) o **`git-cmd.exe`** (para trabajar con Git desde una terminal basada en el intérprete de comandos de Windows).

Una vez descargado (e instalado si procede), **Git se utiliza mediante comandos** en una ventana de terminal.

Vamos a trabajar con la opción en la cual podemos abrir la interfaz de línea de comandos (Shell) de Linux Bash:



Podemos comprobar que el programa se ha instalado correctamente comprobando el número de versión:



Configuración inicial de Git

Antes de usar Git es conveniente emplear unos minutos en configurar la herramienta.

Lo primero que hay que hacer es registrar nuestro nombre y correo electrónico ya que estos datos aparecerán en los cambios (*commits*) que realicemos:

```
tasco@Fortaleza77 UCRT64 /
$ git config --global user.name "Luis Javier Menendez"

tasco@Fortaleza77 UCRT64 /
$ git config --global user.email "luisjmt@educastur.org"

tasco@Fortaleza77 UCRT64 /
```

Estos datos identifican al autor de los commits, pero no sirven para autenticarse en GitHub.

El parámetro `--global` hace que esta configuración se aplique a todos los proyectos en tu ordenador. Puedes omitirlo si quieres que los datos sean diferentes para cada proyecto.

Cuando trabajemos con GitHub y necesitemos realizar operaciones que requieran autenticación (por ejemplo, subir cambios mediante `git push`), deberemos identificarnos de forma segura. Para facilitar este proceso utilizaremos **Git Credential Manager**, una herramienta que permite autenticarnos mediante el navegador y gestionar las credenciales para evitar tener que identificarnos repetidamente.

Para configurar Git Credential Manager como gestor de credenciales, ejecutamos:

```
tasco@Fortaleza77 UCRT64 /
$ git config --global credential.helper manager

tasco@Fortaleza77 UCRT64 /
$
```

Esto guarda de forma segura tus datos de acceso para futuras sesiones.

NOTA: En algunas instalaciones antiguas de Git puede estar configurado el gestor de credenciales `wincred`. Aunque permite almacenar credenciales en Windows, no ofrece las mismas posibilidades de autenticación moderna que Git Credential Manager.

Si encontramos problemas de autenticación al ejecutar `git push` o `git pull`, debemos comprobar qué gestor tenemos configurado:

```
tasco@Fortaleza77 UCRT64 /
$ git config --global --get credential.helper
manager
```

Si aparece `wincred`, podemos cambiarlo por `manager`:

```
git config --global credential.helper manager
```

Para utilizarlo, Git Credential Manager debe estar disponible en nuestro equipo. Para comprobarlo, podemos ejecutar:

```
tasco@Fortaleza77 UCRT64 /
$ git credential-manager --version
2.9.1+6760f0ef069c994aa2bb1d703fb374986ee82a3e
```

Es posible comprobar estas, y otras configuraciones, con la opción *--list*:

```
tasco@Fortaleza77 UCRT64 /
$ git config --list
core.symlinks=false
core.autocrlf=true
core.fscache=true
color.interactive=true
color.ui=auto
help.format=html
diff.astextplain.textconv=astextplain
rebase.autosquash=true
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
credential.helper=helper-selector
user.name=Luis Javier Menendez
user.email=luisjmt@educastur.org
credential.helper=manager
```

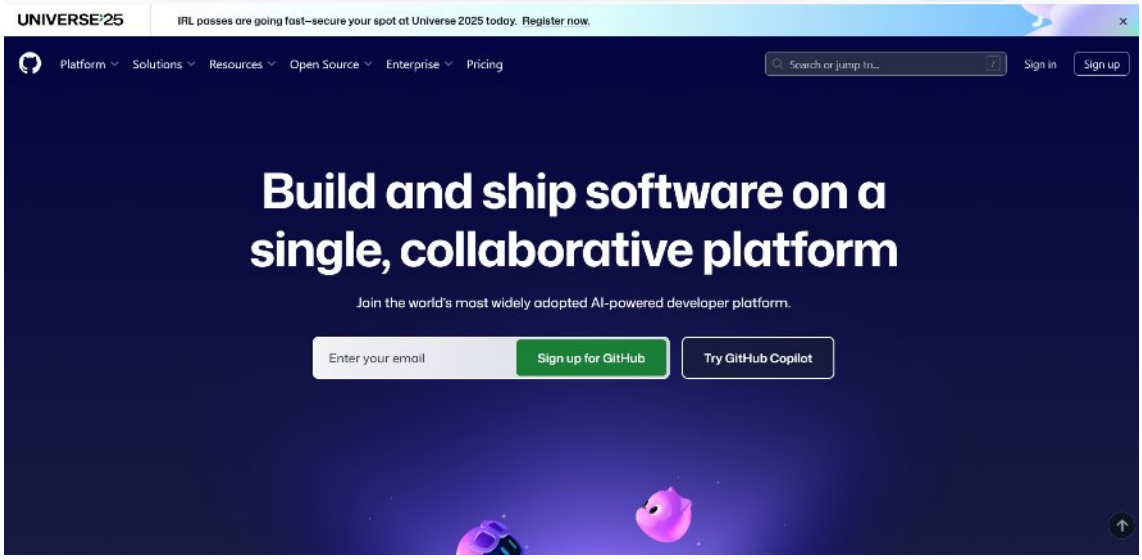
Aunque existen entornos visuales muy cómodos para Git (como GitHub Desktop, SourceTree o GitKraken), te recomiendo acostumbrarte a la terminal.

Aprender Git con comandos te permitirá entender mejor lo que ocurre "detrás del botón", resolver problemas con más autonomía y desenvolverte en cualquier entorno profesional. Los auténticos programadores no le temen a la línea de comandos.

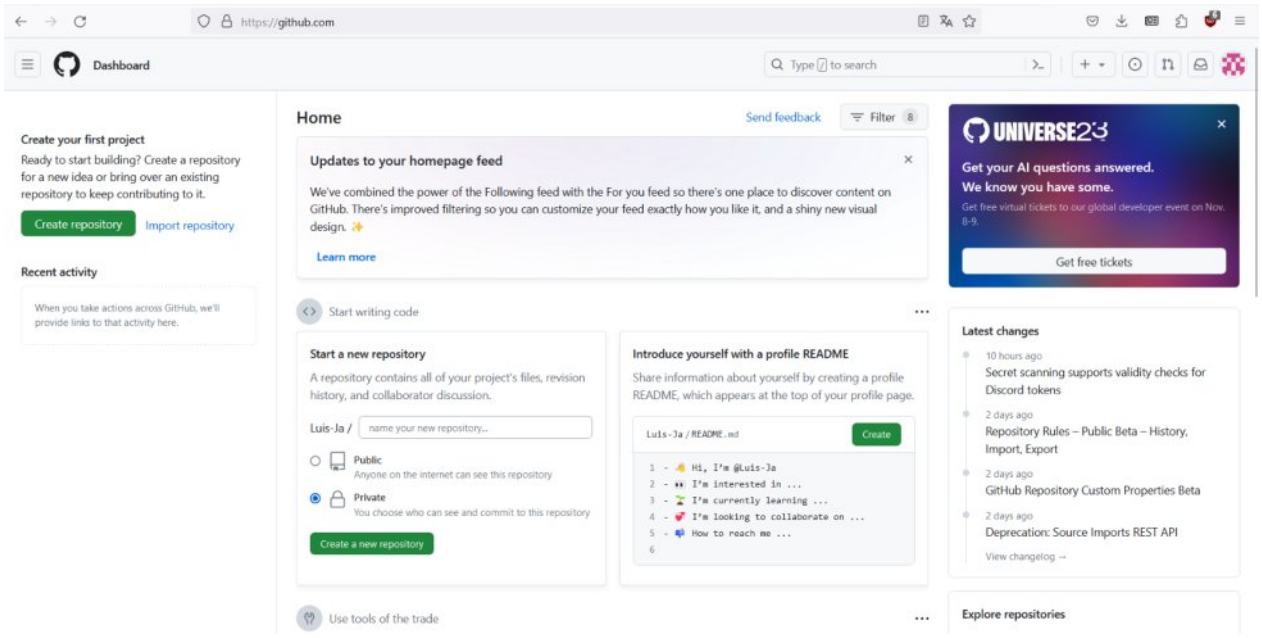
2.2. En cuanto a GitHub

Lo primero que tenemos que hacer es crearnos una cuenta en GitHub.

Entra en la web de GitHub (https://github.com) y haz clic en el botón *Sign up*:



Una vez introducidos los datos necesarios (nombre de usuario, correo electrónico, contraseña, etc.), tendremos que verificar nuestra dirección de correo electrónico. También es posible que GitHub nos solicite configurar la autenticación en dos pasos (2FA), un mecanismo adicional de seguridad para proteger nuestra cuenta. Después accederemos al *Dashboard* inicial, donde veremos nuestro perfil y las opciones para comenzar a trabajar con proyectos.



ANEXO - Autenticación de dos factores (2FA)

Fuente: Incibe

La verificación en dos pasos, también conocida como autenticación de dos factores (2FA), doble factor o similar, es una medida de seguridad adicional a la contraseña que se utiliza para proteger del acceso no autorizado a las cuentas de los usuarios online.

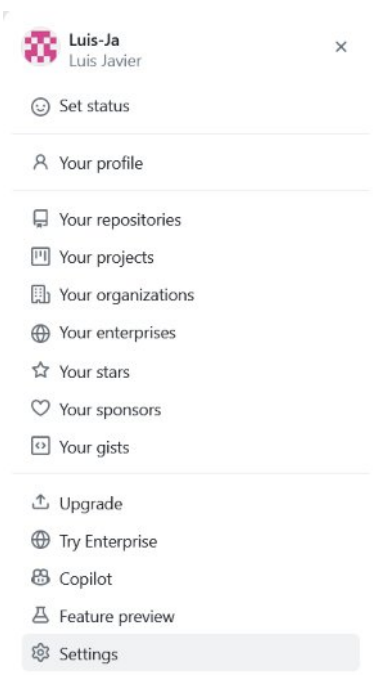
Por lo general, la verificación en dos pasos se realiza en dos etapas:

- Primer paso, introducir el primer factor: Algo que el usuario sabe, como una contraseña o un código PIN.
- Segundo paso, facilitar el segundo factor: Algo que el usuario posee, como es un dispositivo móvil, una llave física, un dispositivo USB, etc. al que se envía o permite generar un código único temporal conocido como OTP por sus siglas en inglés (*One-Time Password*) que habrá que introducir en el proceso de *login.*

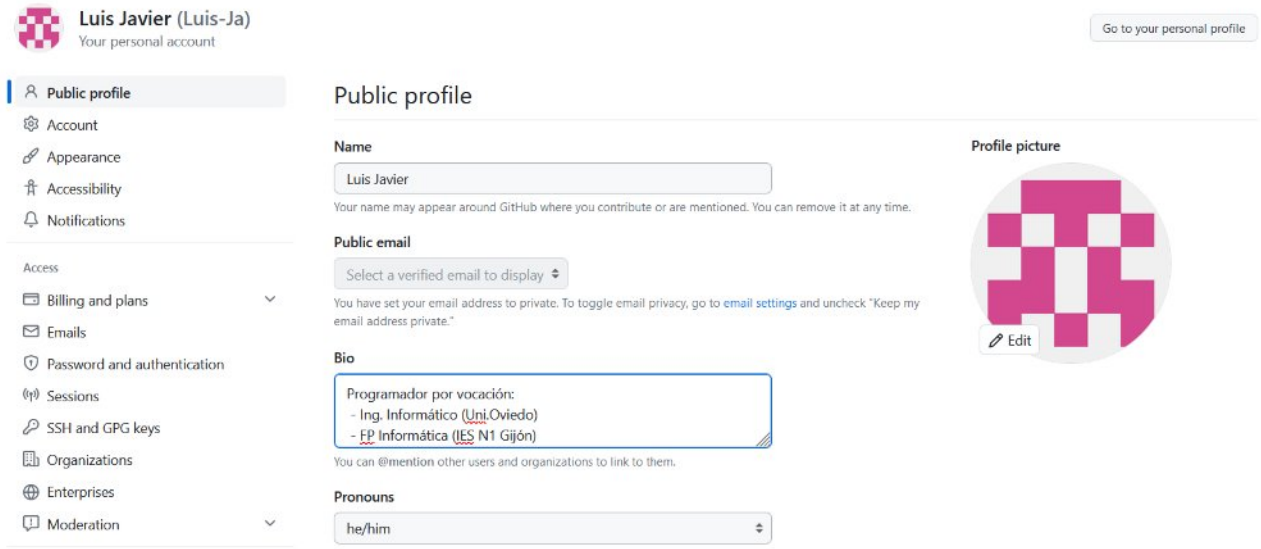
El uso de estos dos factores hace que sea mucho más difícil para los atacantes acceder a una cuenta, incluso si obtienen o adivinan la contraseña. La idea es que, si un factor de autenticación es comprometido, que generalmente suele ser el primero, el otro factor sigue siendo necesario para acceder a la cuenta, y por lo tanto, no podrían acceder a tu cuenta de usuario.

Configuración inicial de nuestro perfil en GitHub

Antes de realizar cualquier acción dentro de GitHub es conveniente realizar unos ajustes previos. En la esquina superior derecha, haz clic en el icono de tu avatar (inicialmente se asigna un avatar aleatorio por defecto). En el menú desplegable que aparece selecciona "*Settings*":



De esta manera accedes a tu página de perfil:



Podemos cambiar el Avatar por una foto más personal (siempre con ojo, hay muchos perfiles de buenos programadores en GitHub estropeados por una mala foto).

Escribe tu nombre completo (nombre y apellidos), con los correspondientes espacios, mayúsculas y tildes si procede. Anota en el campo *Bio* cualquier curso que hayas hecho o estés haciendo relacionado con la programación o con la informática en general, en orden de importancia. Por ejemplo, si estás estudiando un ciclo formativo o un grado universitario, éste es el lugar idóneo para escribirlo: "*Estudiante de DAW en Mieres. Apasionado del desarrollo web y la automatización con Python.*"

Si tienes una web personal o un blog con contenidos relacionados con la programación o con algo que tenga que ver con la informática, escribe la dirección en el campo URL. Si no tienes nada de eso, escribe en este campo la dirección de tu perfil de LinkedIn. En caso de no tener cuenta en LinkedIn, te recomiendo encarecidamente que te des de alta (si no ahora, en un futuro próximo) → es la red social profesional por excelencia, muy práctica para hacer contactos a nivel corporativo y también es muy útil en la búsqueda de empleo.

El campo *Location* es muy importante. Aunque es opcional, puede facilitar que otras personas o empresas encuentren nuestro perfil mediante búsquedas por ubicación. Escribe el nombre de tu ciudad y, entre paréntesis, el nombre de tu país en inglés.

Por último, si quieres decirle al mundo que estás disponible para que te contraten, marca la casilla "*Available for hire*". Hay aplicaciones que sondean los perfiles de GitHub en busca de candidatos para puestos de programación y miran si esta casilla está marcada. Recuerda que tu cuenta de GitHub es tu mejor carta de presentación como programador.



Vamos a realizar ahora una pequeña actividad:

- Entra en GitHub e inicia sesión. Pulsa sobre tu avatar, en la esquina superior derecha.
- Selecciona Your profile. Copia la URL de tu perfil, que tendrá una estructura similar a `https://github.com/NombreUsuario`.
- Abre una ventana de incógnito en el navegador y pega esa dirección, sin iniciar sesión en GitHub.
- Observa qué información aparece: avatar, nombre, biografía, ubicación, repositorios públicos, contribuciones, etc:
  - ¿La información que aparece es correcta y está actualizada?
  - ¿Hay información personal que preferirías no mostrar públicamente?
  - Si fueras una empresa buscando un desarrollador, ¿qué impresión te produciría ese perfil?

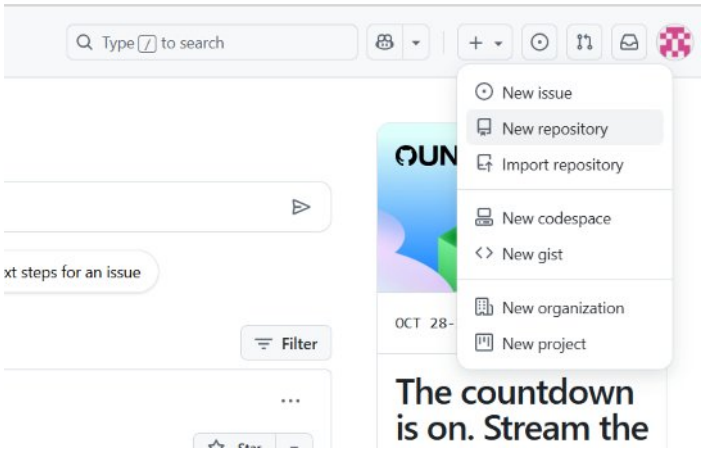
3- Creación de repositorios en GitHub

Un repositorio es el espacio donde se almacenan los archivos de un proyecto y se gestiona su historial de versiones mediante Git. Puede contener código fuente, documentación, imágenes y otros recursos.

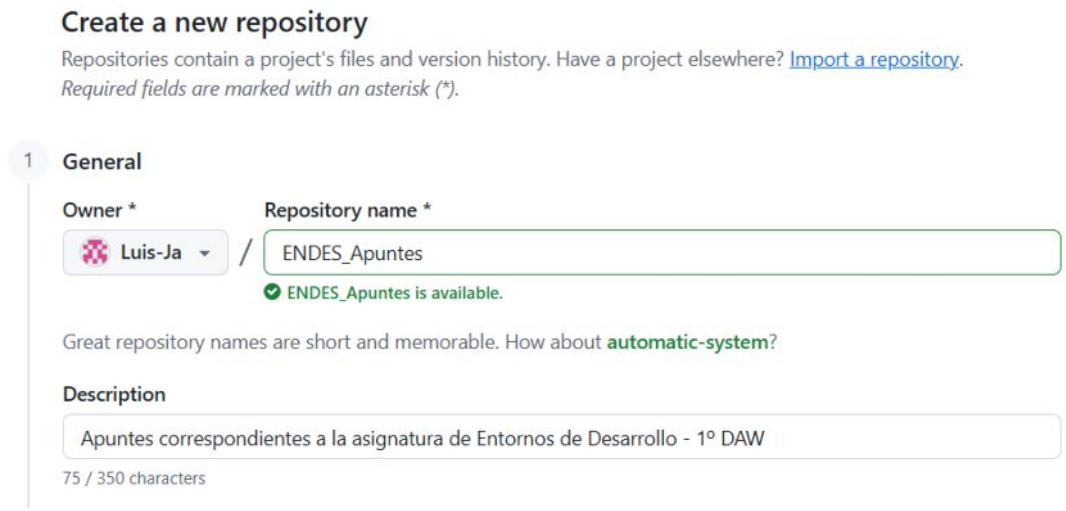
En GitHub, los repositorios se alojan en servidores remotos (en la nube), lo que permite compartirlos y colaborar con otros desarrolladores.

Para crear un nuevo repositorio:

- Haz click en el signo **+** que hay junto a tu avatar.
- Selecciona *New repository* en el menú desplegable:



Escribe el nombre del repositorio en el campo *Repository name*. Debe ser un nombre lo más claro y conciso posible. **No se permiten espacios en blanco ni caracteres especiales en este campo. Las palabras pueden estar separadas por guiones**. Como ejemplo práctico, si el repositorio que estamos creando va a contener nuestros primeros programas en Java, un nombre perfecto puede ser "Ejemplos-java-sencillos":

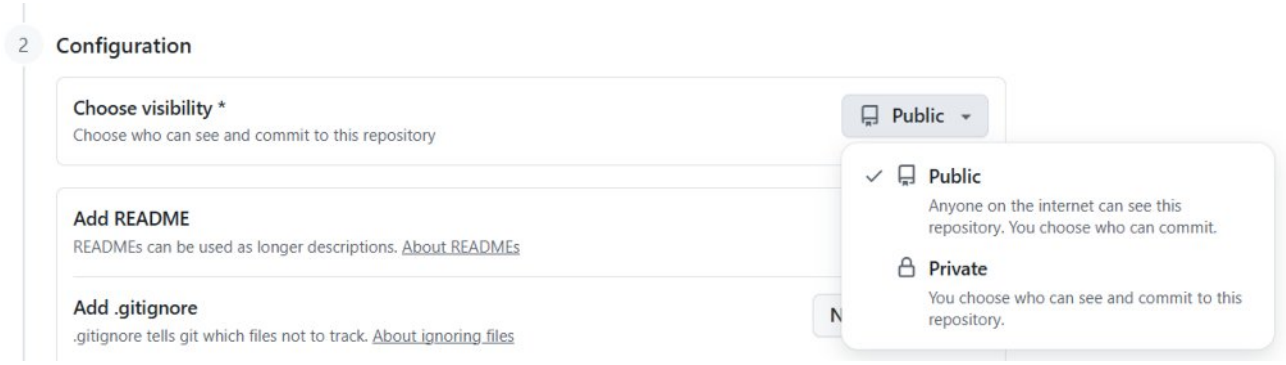


El campo *Description* es opcional pero es muy recomendable rellenarlo porque ayuda a entender el propósito del repositorio, incluso cuando uno vuelve meses después.

Debemos seleccionar la visibilidad del repositorio:

- Public: cualquier usuario de Internet puede consultar su contenido.
- Private: únicamente el propietario y las personas autorizadas pueden acceder a él.

Esta configuración puede modificarse posteriormente.



Inicialización del repositorio: GitHub nos permite crear el repositorio con un archivo inicial `README.md`, que se utiliza habitualmente para describir el proyecto y proporcionar información sobre su contenido.



¿Debemos crear siempre este archivo? No necesariamente. Dependerá de cómo vayamos a comenzar a trabajar con Git y GitHub.

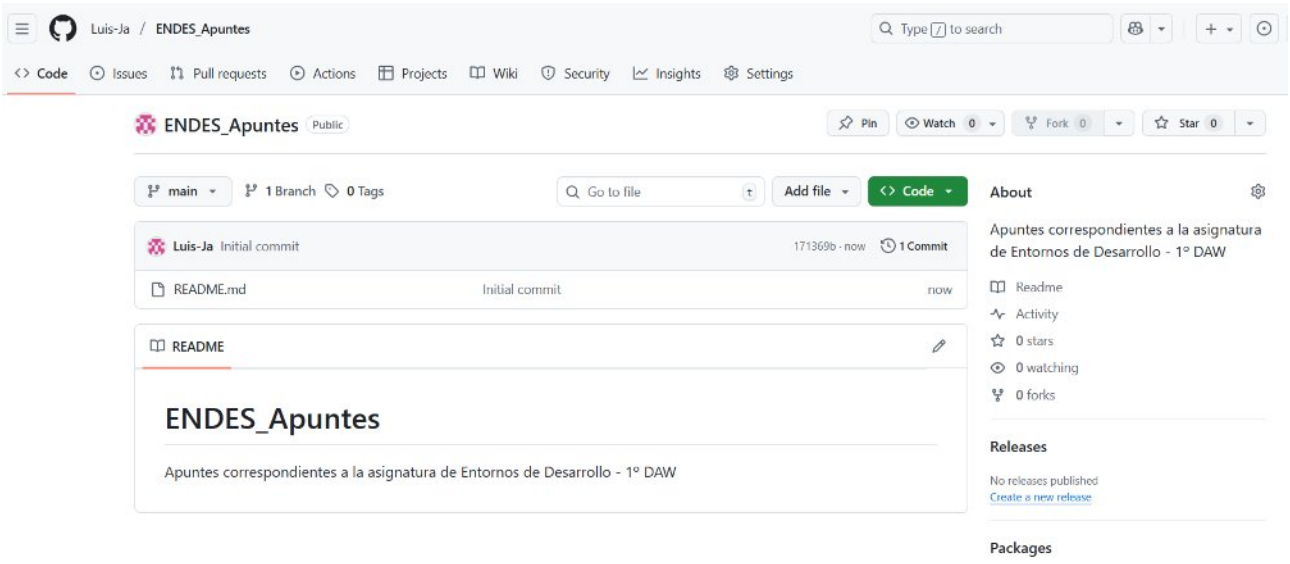
- **Escenario 1 – Partimos de GitHub:** creamos el repositorio incluyendo un README inicial. Posteriormente aprenderemos a clonarlo en nuestro ordenador.
- **Escenario 2 – Partimos de un proyecto local (gestionado por GIT):** creamos el repositorio de GitHub completamente vacío, sin README, `.gitignore` ni licencia, para conectarlo posteriormente con nuestro proyecto local.
- **Escenario 3 – Historiales independientes:** veremos qué ocurre cuando creamos por separado un repositorio local con *commits* y otro en GitHub con README inicial. Aprenderemos a identificar y resolver el problema que puede producirse al intentar integrarlos.

En nuestro primer ejemplo utilizaremos el escenario 1, por lo que activaremos la opción *Add README* antes de crear el repositorio.

Al marcar esta opción, se crea el fichero README.md que contiene por defecto el nombre y la descripción del repositorio; de esta forma ya hay algo dentro del proyecto, no está vacío y, por tanto, ya tenemos algo que clonar a nuestro ordenador (en el siguiente apartado veremos que significa esto).

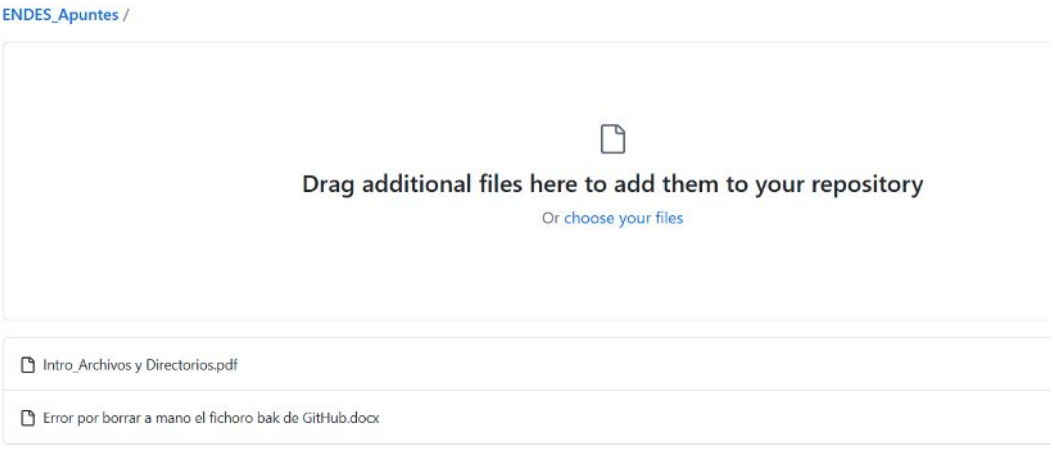
El resto de opciones (*gitignore* o *license*) podemos dejarlas por defecto.

Una vez creado el repositorio, aparece una vista general desde la que se puede ver el contenido del mismo y mucha otra información que iremos analizando más adelante:

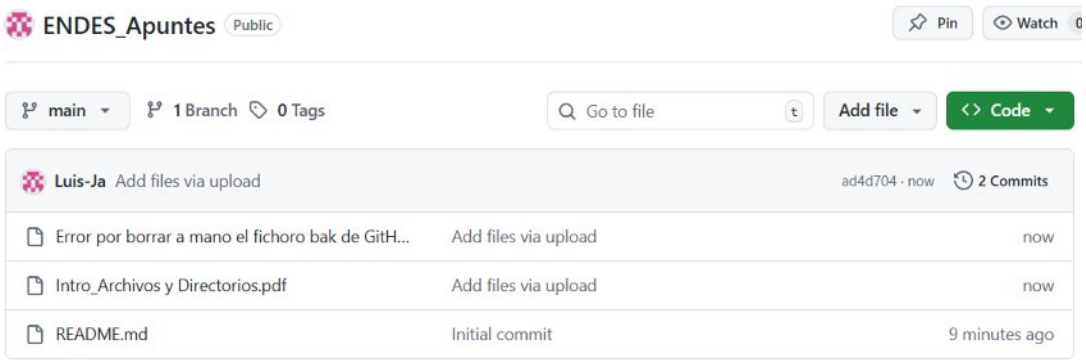


Ya tenemos nuestro primero repositorio recién creado en GitHub.

Puedes añadir archivos directamente desde el botón **"Add file → Upload files"**. Por ejemplo:



Pulsamos el botón de *Commit changes* y los ficheros quedarán subidos a nuestro repositorio:



Y vemos que se almacenan en nuestro repositorio.

OJO: Cuando subimos archivos directamente a GitHub mediante *Upload files* y confirmamos la operación con *Commit changes*, estamos creando un nuevo *commit* en el repositorio remoto. Es decir, los *commits* no se realizan exclusivamente desde Git Bash. También podemos crearlos desde la interfaz web de GitHub.

Más adelante aprenderemos a realizar estos cambios desde nuestro ordenador y a enviarlos posteriormente a GitHub mediante el comando `git push`.

Subir archivos directamente en GitHub es útil para pequeñas pruebas o ejercicios rápidos. Sin embargo, **en proyectos reales trabajaremos desde Git en nuestro ordenador (en LOCAL), y los archivos se subirán al repositorio de GitHub asociado haciendo commits y pushes, lo que nos dará un control total sobre el historial**.